

Examflow

AWS Deployment Guide

A complete, step-by-step guide to putting your hall ticket and attendance system online on Amazon Web Services — written assuming you have **no technical background**. Follow it top to bottom, in order, and copy-paste every grey box exactly as shown.

Your questions, answered up front

Question	Answer
Where do I deploy this?	One AWS server (called EC2) running Docker, plus Route 53 for your domain name (DNS).
Do I need to know Linux or Docker?	No — every command you need is given below to copy-paste exactly. You just need to reach the server over SSH once.
Do I need a domain name?	Yes. Teachers' phone browsers only grant camera access over HTTPS, and HTTPS needs a real domain name pointed at your server.
How do students get emailed?	Through your Gmail / Google Workspace account, using a special "app password" (Part 3).
How much will this cost?	Roughly \$10–20/month — see the cost breakdown in the Appendix. New AWS accounts get 750 free EC2 hours/month for the first 12 months.

How this differs from a "just click deploy" service: AWS doesn't host anything for you automatically — you're renting a bare server and wiring up every piece yourself: the server, the domain, HTTPS, the database, the email sender. That's more steps than a service like Render, but it's also what "deploying on AWS" means, and this guide walks through every single one of those steps with exact commands.

What's in this guide

- 1 Part 0 — What you'll need before starting
- 2 Part 1 — Create your AWS account
- 3 Part 2 — Buy a domain name
- 4 Part 3 — Set up Gmail so emails actually send
- 5 Part 4 — Launch your server (EC2)

- 6 Part 5 — Point your domain at your server (DNS, in depth)
- 7 Part 6 — Connect to your server and install Docker
- 8 Part 7 — Get the project code onto the server
- 9 Part 8 — Configure your environment variables (.env)
- 10 Part 9 — Set up automatic HTTPS
- 11 Part 10 — Launch everything
- 12 Part 11 — First-time setup after deployment
- 13 Part 12 — Exam day checklist
- 14 Part 13 — Backups and keeping it secure
- 15 Part 14 — If something goes wrong
- 16 Appendix A — Monthly cost breakdown
- 17 Appendix B — Command reference (everything in one place)

Part 0 — What you'll need before starting

Gather these first so you're not stopping halfway through:

- A credit/debit card, to create an AWS account (AWS bills monthly; nothing is charged upfront).
- A Gmail address, or better, a Google Workspace account — this is what will send hall ticket emails.
- About **60–90 minutes** of uninterrupted time for the first setup.
- A computer with a terminal (Mac/Linux: built in. Windows: use PowerShell, which has SSH built in since Windows 10).
- This project's code, already on GitHub at `github.com/shubhnb5/exam-management-system`.

Everywhere in this guide, a grey box like the one below is something to copy and paste exactly as shown — don't retype it by hand, small typos will stop things from working.

Example of a copy-paste box

Part 1 — Create your AWS account

1. Go to `aws.amazon.com` and click **Create an AWS Account**.
2. Enter your email, choose an account name (e.g. "Examflow"), and set a password.
3. Enter your card details — AWS requires this even for free-tier usage, but you won't be charged unless you exceed the free tier or use paid resources (this guide uses a small paid server, roughly \$10–20/month — see Appendix A).
4. Verify your phone number when asked (an automated call or SMS with a code).
5. Choose the **Basic support plan** (free) when prompted.
6. Once in, use the search bar at the top of the AWS Console to jump between services — you'll use **EC2** (the server) and **Route 53** (DNS) in this guide.

Set a strong password on this account and enable MFA (Console → top-right account menu → Security credentials → Assign MFA device) — this account can spend real money and holds access to a system with student data.

Part 2 — Buy a domain name

A domain name (like `examflow-college.com`) is the human-readable address for your server. You'll use it to make three addresses: one for the admin dashboard, one for the scanner, and one for the backend.

1. In the AWS Console, search for and open **Route 53**.
2. Click **Domains** → **Register domain**.
3. Search for a name, e.g. `yourcollege-exam.com`. Prices are typically \$10–15/year for a `.com`.
4. Add it to cart, fill in your contact details (used for domain ownership records — enable "Privacy protection", which is free and hides these from public lookup), and complete the purchase.
5. Registration usually finishes in a few minutes, occasionally up to an hour. Route 53 automatically creates a **Hosted Zone** for it — you'll use this in Part 5.

Already own a domain elsewhere (GoDaddy, Namecheap, etc.)? You don't need to buy a new one. In Route 53, go to **Hosted zones** → **Create hosted zone**, enter your existing domain name, create it — then in Part 5

you'll copy 4 "name server" addresses from that hosted zone into your other registrar's settings, which hands DNS control over to Route 53 without moving the domain itself.

Part 3 — Set up Gmail so emails actually send

Gmail won't let this app log in with your normal password — Google requires a special 16-character "App Password" for anything other than a human typing into gmail.com. This takes about 5 minutes.

Step 3.1 — Turn on 2-Step Verification

1. Go to `myaccount.google.com/security`, signed in as the account you'll send emails from.
2. Find "2-Step Verification" and turn it on if it says "Off" (Google will text a code to your phone to confirm).
3. Already "On"? Skip to Step 3.2.

Step 3.2 — Create an App Password

1. Go to `myaccount.google.com/apppasswords` (you may need to sign in again).
2. Under "App name", type **Examflow** and click Create.
3. Google shows a 16-character password like `abcd efgh ijkl mnop`. Copy it down now — Google only shows it once.
4. Remove the spaces when you use it later, so it becomes one word: `abcdefghijklmnop`.

This app password is **not** your Gmail password — it's a separate secret code just for this app. Never share it.

If your account is a school/company Google Workspace account and you don't see the "App Passwords" option, ask your Workspace admin to enable it, or to enable 2-Step Verification first (this option is sometimes restricted by an organization's admin policy).

Sending volume: A standard Gmail account can send ~500 emails/day; a Google Workspace account can send ~2000/day. For a batch of ~1000–1500 students, use a Workspace account, or spread sending across 2–3 days on a plain Gmail account.

Part 4 — Launch your server (EC2)

This is the actual computer that will run the app, 24/7. "EC2" is AWS's name for a rentable server ("instance").

Step 4.1 — Launch the instance

1. In the AWS Console, search for and open **EC2**, then click **Launch instance**.
2. Name: `examflow-server`.
3. Application and OS Image: choose **Ubuntu**, then **Ubuntu Server 24.04 LTS** (marked "Free tier eligible").
4. Instance type: choose **t3.small** (2 GiB RAM). See the note below on why not the free t2.micro/t3.micro.
5. Key pair: click **Create new key pair**, name it `examflow-key`, type **RSA**, format **.pem** (Mac/Linux/PowerShell) — click **Create**, and a file downloads. Keep it safe; you cannot re-download it later.
6. Network settings → click **Edit** and set up these three inbound rules (delete any others):

Type	Port	Source
SSH	22	My IP (auto-filled by AWS)
HTTP	80	Anywhere (0.0.0.0/0)
HTTPS	443	Anywhere (0.0.0.0/0)

7. Configure storage: change the default to **20 GiB**, type **gp3**.
8. Click **Launch instance**.

Why not the free t2.micro? It only has 1 GiB RAM, which can run out of memory while building the two frontend websites in Part 10. t3.small costs about \$15/month — worth it for an exam-critical system. If you want to try the free tier anyway, add a 2 GiB swap file first (commands in Appendix B).

Step 4.2 — Give it a fixed address (Elastic IP)

A normal EC2 instance's public IP address can change if it's ever stopped and restarted. An **Elastic IP** is a fixed address you own, so your domain always points to the right place.

1. In the EC2 left sidebar, click **Elastic IPs** → **Allocate Elastic IP address** → **Allocate**.
2. Select the new address → **Actions** → **Associate Elastic IP address**.
3. Resource type: Instance. Pick `examflow-server`. Click **Associate**.
4. Write this IP address down (e.g. `13.234.56.78`) — you'll use it repeatedly.

An Elastic IP is free *while attached to a running instance*, but AWS charges a small hourly fee if it's ever left unattached — don't allocate one you don't plan to use.

Part 5 — Point your domain at your server (DNS, in depth)

DNS (Domain Name System) is the phonebook of the internet — it maps a human-readable name (`admin.yourdomain.com`) to a numeric address (your Elastic IP). This section goes through it in full, since it's the part most likely to feel unfamiliar.

Step 5.1 — Decide your three addresses

You'll create three separate addresses under your one domain, called **subdomains** — each one will point at the same server, but Part 9's HTTPS setup uses the name to route traffic to the right piece of the app inside:

Subdomain	What it's for
<code>admin.yourdomain.com</code>	The admin dashboard (you use this)
<code>scan.yourdomain.com</code>	The scanner PWA (teachers use this on their phones)
<code>api.yourdomain.com</code>	The backend — the two websites above talk to this automatically; you won't visit it directly except to test

Step 5.2 — Create the DNS records

1. In the AWS Console, open **Route 53** → **Hosted zones**, and click your domain.
2. Click **Create record**.
3. Record name: `admin` (this becomes `admin.yourdomain.com`).
4. Record type: **A** — "A" means "this name points directly to an IPv4 address" (as opposed to a CNAME, which points to another name — you don't need that here).
5. Value: paste your Elastic IP from Step 4.2, e.g. `13.234.56.78` .
6. TTL (Time To Live): set to **300** seconds (5 minutes) — this controls how long other computers are allowed to "remember" this answer before checking again. A short TTL is useful while you're setting things up, since any correction you make shows up faster. You can raise it to a few hours later once everything is stable.
7. Click **Create records**.
8. Repeat the same steps twice more, for `scan` and `api` — same type (A), same value (your Elastic IP), same TTL.

You should end up with 3 new "A" records in the hosted zone: `admin.yourdomain.com` , `scan.yourdomain.com` , `api.yourdomain.com` — all pointing at the same IP.

Step 5.3 — If your domain was bought outside Route 53

Skip this step if you bought the domain directly through Route 53 in Part 2 — it's already wired up correctly. If you registered elsewhere (GoDaddy, Namecheap, etc.) and created a Hosted Zone for it in Route 53 instead:

1. Open your hosted zone in Route 53 — there's a record of type **NS** (Name Server) at the root, listing 4 addresses like `ns-123.awsdns-45.com` .
2. Log into your domain's original registrar (GoDaddy/Namecheap/etc.), find **DNS settings** → **Nameservers**, and replace whatever is there with those exact 4 addresses from Route 53.
3. This step is called **delegation** — it tells the rest of the internet "for this domain, go ask Route 53 for the answer" instead of the registrar's own DNS.

4. This can take anywhere from 15 minutes to 24–48 hours to fully take effect worldwide (see propagation, below).

Step 5.4 — Verify it's working

DNS changes don't appear instantly everywhere — this is called **propagation**: every computer/ISP on the internet caches answers for the TTL duration before re-checking. To verify your records:

- **Mac/Linux terminal or Windows PowerShell:** run `nslookup admin.yourdomain.com` — it should print your Elastic IP under "Address". Repeat for `scan.` and `api.`
- **No terminal handy:** use a browser-based checker like `dnschecker.org`, type in `admin.yourdomain.com`, and it shows you what different locations around the world currently see.

Common pitfalls:

- Forgetting Step 5.3 (nameserver delegation) if the domain was bought elsewhere — without it, Route 53's records are never consulted at all.
- Your own laptop/browser caching an old (wrong) answer — if a site was unreachable and now should work, try a different network (phone hotspot) or an incognito/private window before assuming DNS itself is broken.
- Testing immediately after creating a record — give it at least the TTL duration (5 minutes) before concluding something's wrong.

You do not need to touch anything on the EC2 side for DNS — Part 9's HTTPS setup is what actually makes the server respond to these names once traffic arrives.

Part 6 — Connect to your server and install Docker

Step 6.1 — Connect over SSH

1. Open a terminal (Mac/Linux: Terminal app. Windows: PowerShell).
2. Move into the folder where your `examflow-key.pem` file downloaded, then lock down its permissions (Mac/Linux only — skip on Windows):

```
chmod 400 examflow-key.pem
```

3. Connect, replacing the IP with your Elastic IP from Step 4.2:

```
ssh -i "examflow-key.pem" ubuntu@YOUR_ELASTIC_IP
```

4. Type `yes` if asked about the host's fingerprint the first time.

Stuck? On the EC2 instance page in the AWS Console, click the orange **Connect** button and choose the **EC2 Instance Connect** tab for a browser-based terminal — no key file needed, as a fallback.

Step 6.2 — Install Docker

Once connected (your prompt now shows `ubuntu@ip-...`), paste this one block — it installs Docker and Docker Compose together:

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker $USER
exit
```

1. After it finishes, you'll be disconnected (that's expected — `exit` logs you out so the permission change takes effect). SSH back in with the same command as Step 6.1.
2. Check it worked:

```
docker --version
docker compose version
```

Both should print a version number, not an error.

Part 7 — Get the project code onto the server

Still connected over SSH:

```
git clone https://github.com/shubhnb5/exam-management-system.git examflow
cd examflow
```

If the repository is private, git will prompt for a username and password — use your GitHub username and a **Personal Access Token** (GitHub.com → Settings → Developer settings → Personal access tokens → generate

one with "repo" scope) in place of the password.

Part 8 — Configure your environment variables (.env)

The `.env` file holds every secret and setting the app needs — the database password, the email login, the admin login. It's never committed to GitHub, so you create it directly on the server.

Step 8.1 — Generate two random secrets

Still in the `examflow` folder on the server, run these two commands and copy down each result — you'll paste them into the file in Step 8.2:

```
echo "JWT secret: $(openssl rand -base64 32)"
echo "Admin password: $(openssl rand -base64 12)"
echo "DB password: $(openssl rand -hex 16)"
```

Write all three down now (copy them somewhere safe, e.g. a password manager) — you'll need the admin password again in Part 11 to log in.

Your credentials at a glance — fill this in as you go, and keep it somewhere safe (a password manager, not this PDF):

Credential	Where it's used	Where it comes from
Admin dashboard password	Logging into <code>admin.yourdomain.com</code> (Part 11.1)	Generated in Step 8.1, set as <code>ADMIN_DEFAULT_PASSWORD</code>
Database password	Internal only — the app uses it automatically, you never type it anywhere	Generated in Step 8.1, set as <code>POSTGRES_PASSWORD</code>
JWT secret	Internal only — signs login sessions, never shown to a user	Generated in Step 8.1, set as <code>JWT_SECRET_KEY</code>
Gmail app password	Backend uses it to send ticket emails	From Part 3.2, set as <code>SMTP_APP_PASSWORD</code>
SSH key file (<code>examflow-key.pem</code>)	Connecting to the server (Part 6.1)	Downloaded once in Step 4.1 — cannot be re-downloaded
Each teacher's login	Scanner PWA at <code>scan.yourdomain.com</code>	You choose these yourself in Part 11.2, one per teacher/device

Step 8.2 — Create the .env file

Run this command, then replace every `<...>` placeholder with your real values before saving (it opens the `nano` text editor — arrow keys to move, type to edit, `Ctrl+O` then `Enter` to save, `Ctrl+X` to exit):

```
nano .env
```

Paste this in (select all and delete anything already there first), then edit the placeholders:

```
# --- Database ---
POSTGRES_USER=examflow
```

```
POSTGRES_PASSWORD=<paste the DB password from Step 8.1>
POSTGRES_DB=examflow
DATABASE_URL=postgresql+psycopg2://examflow:examflow@localhost:5432/examflow

# --- Auth ---
JWT_SECRET_KEY=<paste the JWT secret from Step 8.1>
JWT_ALGORITHM=HS256
JWT_EXPIRE_MINUTES=720

# --- Email (Gmail / Google Workspace) ---
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USERNAME=<your Gmail/Workspace address from Part 3>
SMTP_APP_PASSWORD=<the 16-character app password from Part 3, no spaces>
SMTP_FROM_NAME=<the name students should see as the sender, e.g. Examflow>

# --- Ticket storage ---
TICKET_STORAGE_DIR=./storage/tickets

# --- App ---
ADMIN_DEFAULT_USERNAME=admin
ADMIN_DEFAULT_PASSWORD=<paste the admin password from Step 8.1>

# --- Used to build the two websites with the correct backend address ---
PUBLIC_API_URL=https://api.yourdomain.com
```

Leave `DATABASE_URL` 's `examflow:examflow@localhost` part exactly as shown — `docker-compose.yml` automatically rebuilds the real database connection from your `POSTGRES_*` values above when it starts the containers; you don't need to edit that line by hand.

Replace `yourdomain.com` in `PUBLIC_API_URL` with your real domain from Part 2 — this must exactly match, with no trailing slash.

Part 9 — Set up automatic HTTPS

This is the piece that turns "a server with an IP address" into " `https://admin.yourdomain.com` with a padlock icon" — required for the scanner's camera to work on real phones. We use a small extra piece of software called **Caddy**, which automatically obtains and renews a free HTTPS certificate (from Let's Encrypt) for each domain name you give it.

Step 9.1 — Create the Caddy configuration file

Still in the `examflow` folder, run:

```
cat > Caddyfile <<'EOF'
admin.yourdomain.com {
    reverse_proxy frontend-admin:80
}

scan.yourdomain.com {
    reverse_proxy frontend-scanner:80
}

api.yourdomain.com {
    reverse_proxy backend:8000
}
EOF
```

Before running that command, replace all three `yourdomain.com` occurrences with your real domain — easiest to paste the block into `nano Caddyfile` instead, edit it there, then save (`Ctrl+O` , `Enter` , `Ctrl+X`).

Step 9.2 — Add Caddy to docker-compose.yml

Open the compose file:

```
nano docker-compose.yml
```

Add this new block anywhere inside `services:` (matching the indentation of `backend:` , `frontend-admin:` , etc. — 2 spaces in):

```
caddy:
  image: caddy:2-alpine
  restart: unless-stopped
  depends_on:
    - backend
    - frontend-admin
    - frontend-scanner
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./Caddyfile:/etc/caddy/Caddyfile
    - caddy_data:/data
    - caddy_config:/config
```

Then find the `volumes:` section at the very bottom of the file (it currently just has `db_data:`) and add two more lines under it:

```
volumes:
  db_data:
  caddy_data:
  caddy_config:
```

Save (`Ctrl+O` , Enter) and exit (`Ctrl+X`).

You don't need to remove or change the existing `ports:` lines under `backend` / `frontend-admin` / `frontend-scanner` — your AWS Security Group from Step 4.1 only opens 22, 80, and 443 to the internet, so those other ports (8000, 8080, 8081) simply aren't reachable from outside the server anyway. Caddy is what the outside world actually talks to.

Part 10 — Launch everything

Where does the database come from? You don't create it separately — it's the `db` entry already defined in `docker-compose.yml` (Postgres, running in its own container on this same server, storing its files in a Docker volume called `db_data` so they survive restarts). The one command below starts it alongside everything else, using the `POSTGRES_USER` / `POSTGRES_PASSWORD` / `POSTGRES_DB` values you set in Part 8. The backend then automatically creates all the actual tables (students, tickets, scans, etc.) the first time it starts, via a migration step built into its startup command — there is nothing for you to run by hand.

```
docker compose up --build -d
```

1. This builds and starts all 5 containers — **db** (the database), **backend** (the API, which also creates the database tables on this first run), **frontend-admin**, **frontend-scanner**, and **caddy** (HTTPS) — the first run takes 3–6 minutes.
2. Check everything is running:

```
docker compose ps
```

All rows should say `running` or `Up` . Caddy needs your DNS from Part 5 to already be resolving correctly before it can obtain HTTPS certificates — if you just created those records, give it 5–10 minutes and check logs if needed:

```
docker compose logs caddy --tail=50
```

3. Visit `https://api.yourdomain.com/health` in a browser — you should see `{"status":"ok"}` with a padlock icon.
4. Visit `https://admin.yourdomain.com` — you should see the login page.

Part 11 — First-time setup after deployment

Step 11.1 — Log in

1. Open `https://admin.yourdomain.com`.
2. Log in with username `admin` and the admin password you generated in Step 8.1.

Step 11.2 — Create teacher accounts

1. Go to "Teacher Devices / Accounts" → "Add Teacher".
2. For each teacher/device: enter a username, their full name, a password, and pick their exam center. Repeat for every teacher across every center.

Step 11.3 — Do a small test run first

1. Prepare a tiny Excel sheet with 2–3 of your own email addresses (columns: `Student Name`, `Email`, `Mobile Number`).
2. Upload it, pick a center, click **Generate Hall Tickets**, then **Send Emails**.
3. Confirm the email arrives (check Spam too) with a working PDF and scannable QR code — before uploading real student data.

Step 11.4 — Upload your real students

1. Upload your real Excel sheet(s) — one per exam center, picking the correct center each time.
2. Click **Generate Hall Tickets**, then **Send Emails** — both are safe to leave running in the background; the dashboard shows live progress.
3. Check the Students table for anyone marked "Failed" and click Send Emails again to retry (it automatically skips anyone already sent).

Part 12 — Exam day checklist

- Each teacher opens `https://scan.yourdomain.com` on their own phone's browser.
- They log in with their own username/password from Step 11.2.
- The browser asks for camera permission — they must tap **Allow** (this only works because the address starts with `https://`).
- Students bring their printed/emailed hall ticket; the teacher points the camera at the QR code.
- Green screen = checked in successfully. Red screen = already scanned today, or wrong exam center — the on-screen message explains which.
- You (admin) can watch live attendance numbers from anywhere, on any device, at `https://admin.yourdomain.com`.

Part 13 — Backups and keeping it secure

Step 13.1 — Back up the database nightly

This creates a compressed backup file inside the server every night at 2 AM. On the server:

```
mkdir -p ~/backups
crontab -e
```

(First time, it may ask which editor — choose `nano` .) Add this line at the bottom, then save and exit:

```
0 2 * * * cd ~/examflow && docker compose exec -T db pg_dump -U examflow examflow | gzip >
~/backups/examflow-$(date +%Y%m%d).sql.gz
```

Optional — off-server backups: for extra safety, copy backups to an S3 bucket too (Console → S3 → Create bucket, then `aws s3 cp ~/backups/ s3://your-bucket/ --recursive` — the AWS CLI comes pre-installed on the Ubuntu AMI; you'll need to run `aws configure` once with an IAM access key first).

Step 13.2 — Keep the server updated

```
sudo apt update && sudo apt upgrade -y
```

Run this every few weeks. It applies security patches to the underlying Ubuntu system (Docker containers update separately, when you re-run `docker compose up --build -d` after pulling new code with `git pull`).

Step 13.3 — Security basics already in place

- Only ports 22 (SSH), 80, and 443 are open to the internet (Part 4.1) — the database and raw backend/frontend ports are not reachable from outside.
- All traffic is encrypted end-to-end once HTTPS is live (Part 9).
- The one-scan-per-student-per-day rule is enforced by the database itself, not just the app, so it holds even under concurrent scans from different devices.

Change the admin password from the initial one if you ever suspect it's been seen by someone else — update `ADMIN_DEFAULT_PASSWORD` only sets the password on first-ever creation, so to change an existing admin password, do it from inside the dashboard's account settings (or ask for help resetting it directly in the database).

Part 14 — If something goes wrong

Problem	What to do
Browser shows "This site can't be reached"	Check DNS first (Part 5.4 — <code>nslookup admin.yourdomain.com</code> should show your Elastic IP). If DNS is correct, SSH in and run <code>docker compose ps</code> to confirm containers are running.
Browser shows a certificate/HTTPS warning	Caddy hasn't finished obtaining its certificate yet — this needs DNS to be fully resolving first. Wait 5–10 minutes after DNS propagates, then check <code>docker compose logs caddy --tail=50</code> for errors.
Emails aren't sending / show "Failed"	Double check <code>SMTP_USERNAME</code> and <code>SMTP_APP_PASSWORD</code> in <code>.env</code> match Part 3 exactly (no spaces in the app password), then restart: <code>docker compose up -d --build backend</code>
Camera won't open on a teacher's phone	Confirm they're using the <code>https://scan.yourdomain.com</code> address (not typed manually with a typo), and tapped Allow when prompted for camera access. Try closing and reopening the browser tab.
Dashboard loads but shows errors / blank data	Check <code>PUBLIC_API_URL</code> in <code>.env</code> matches your real <code>api.</code> address exactly, then rebuild: <code>docker compose up -d --build</code>
Ran out of memory while building (server froze/crashed)	You're likely on a t2.micro/t3.micro — either resize to t3.small (EC2 console → stop instance → Actions → Instance settings → Change instance type), or add a swap file (Appendix B).
Something else / not covered here	Come back to this Claude Code conversation, describe exactly what you're seeing (and paste any <code>docker compose logs</code> output), and Claude can look at it with you.

Appendix A — Monthly cost breakdown

Item	Approx. cost
EC2 t3.small (2 vCPU, 2 GiB RAM), running 24/7	~\$15/month (free for 750 hrs/month in your first 12 months on t2.micro/t3.micro instead, if you choose that size)
20 GiB gp3 EBS storage	~\$1.60/month
Elastic IP (while attached to a running instance)	\$0
Route 53 hosted zone	\$0.50/month
Route 53 DNS queries	Negligible at this scale (a few cents/month)
Domain name registration	~\$10–15/year (~\$1/month)
Data transfer out (emails, PDFs, page loads)	Negligible at ~1,500 students
Typical total	~\$17–20/month (or ~\$3–5/month during your AWS free-tier first year, on a free-tier instance size)

Set a budget alert: AWS Console → Billing → Budgets → Create budget, e.g. \$25/month, to get an email if costs run higher than expected.

Appendix B — Command reference

Everything from this guide, in one place, for quick copy-paste on a future visit.

Connect

```
ssh -i "examflow-key.pem" ubuntu@YOUR_ELASTIC_IP
```

Check status / view logs

```
cd examflow
docker compose ps
docker compose logs backend --tail=100
docker compose logs caddy --tail=100
```

Pull new code and redeploy

```
cd examflow
git pull
docker compose up -d --build
```

Restart everything

```
docker compose restart
```

Manual database backup (run any time)

```
docker compose exec -T db pg_dump -U examflow examflow | gzip > ~/backups/manual-$(date +%Y%m%d-%H%M).sql.gz
```

Add a 2 GiB swap file (only needed on t2.micro/t3.micro)

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```